

8.5

1. 程序的执行:  $\left\{ \begin{array}{l} \text{编译} \rightarrow \text{计算机能读懂} \\ \text{解释} \rightarrow \text{用程序运行} \end{array} \right.$

2. C语言运行:  $\left\{ \begin{array}{l} \text{① 编辑器 + 编译器} \\ \text{② IDE (集成开发环境)} \end{array} \right.$

用 (Dev C++)  
文件 → 新建 → 源代码

3. 学函数前, 代码框架:

```
#include <stdio.h>

int main()
{
    (光标) ✱
    return 0;
}
```

4. eg1: printf("Hello world!\n");

" " 内的东西为字符串, "\n" 表示需要在输出结果后换一行

eg2: printf(" %d", 12+34); → 输出的是 46

printf(" 12+34=%d", 12+34); 输出 12+34=46

5. eg3: int price=0; → 为一个变量, 用于存放数据。

赋值: 变量初始化

类型 变量  
(int) (price)

变量的名字是一种标识符, 只能由字母、数字、下划线组成  
且开头不能是数字。

(C语言的关键字不能做标识符)

(变量的类型必须确定, 且程序运行中不能更改)

8-6

ANSI C 要在开头定义所有变量  
C99 可以在之后定义

## 1. 读(整数)

输入 `scanf`  
输出 `printf`

`scanf("%d", &price)`

(将值赋给 price)

一定记得要加在所读变量前

(注意, `scanf` 字符串 " " 中出现的东(包括空格)是必须要我们输入的)

2. `const int Amount = 100`; 将数字 100 直接量  $\Rightarrow$  有意义的常量 Amount.

`const` 表示这个变量的值一旦被赋予就不能再修改了.

`const` 属性的变量一般全大写字母

## 3. C语言两个整数运算的结果也只能是(整数), C语言中 10 与 10.0 是不同的

eg:  $10 \neq 10.0$ , 10.0 为浮点数

浮点数即为带小数点的值

(C语言) 计算时会直接舍去小数点后的

当整数与浮点数一起运算, C语言会得到浮点数运算结果, 同时要将 `%d`  $\Rightarrow$  `%f` (会把整数换成浮点数来算)

4. `int` 是整型变量

`double` 双精度浮点数  $\rightarrow$  用 `scanf` 时输入要用 `%lf`

`float` 单精度浮点数

整数值 - `int`  $\left\{ \begin{array}{l} \text{printf}("%d", ...) \\ \text{scanf}("%d", ...) \end{array} \right.$

定义类型

带小数点的值 - `double`

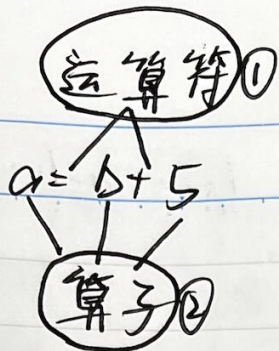
`printf("%f", ...)`

`scanf("%lf", ...)`

# 运算符优先级

8.7

1. eg:



(注意单目)

|   |   |      |              |      |
|---|---|------|--------------|------|
| 1 | + | 单目不变 | eg: $a * +b$ | 自右向左 |
| 1 | - | 单目取负 | eg: $a * -b$ | 自右向左 |
| 2 | * | 乘    |              |      |
| 2 | / | 除    |              |      |
| 2 | % | 取余   |              |      |
| 3 | + | 加    |              |      |
| 3 | - | 减    |              |      |
| 4 | = | 赋值   | (优先级最低) 自右向左 |      |

8.8

2. eg:  $a=5, b=6$ , 如何实  $a, b$  值的交换?

⇒ 引入新变量  $c, \Delta$ , 用于存储数据再交换.

3. 复合赋值

eg:  $total = total + 5 \Leftrightarrow total += 5$

$total *= sum + 12 \Leftrightarrow total = total * (sum + 12)$

4. 递增递减运算符

运算符:  $++$ ,  $--$  为单目运算符.

且算子必须是变量而非数值

作用: 给变量  $+1$  或  $-1$ .

$a++ \rightarrow a$  加  $-1$  之前的值.  
 $++a \rightarrow a$  加  $-1$  之后的值. (但  $a$  本身都会被  $+1$ )

8.9

5. 将十进制转为十六进制:

$\# scanf("%d", &x); \rightarrow$  输入任意十进制整数.  
 $printf("%x", x); \rightarrow$  输出该数的十六进制.

8.11 条件语句 if

框架:  $if(\text{条件成立}) \{ \dots \}$

相等  $==$  不相等  $!=$   
 大于或等于  $>=$   
 小于或等于  $<=$   
 条件关系  
 语法

**关系运算**的结果只有0和1:

eg: `printf("%d", 5 == 3) → 0`

`printf("%d", 5 > 3) → 1`

总之, 关系成立输出1, 不成立输出0。

**关系符号优先级**: 低于算术符号, 高于赋值运算。

eg: `7 >= 3 + 4; int t = a > 0`

↓  
优先

↓  
优先

即: 算术 > 关系 > 赋值

另外: ==, != 的优先级 小于 其他关系符号。

eg: `5 > 3 == 6 > 4; 6 > 5 > 4;`

↓      ↓  
1 == 1 ✓  
↓  
1

↓      ↓  
1 > 4 ×  
↓  
0

(从左往右)

**注释** C99用: <sup>①</sup> `//` 双斜杠 → 单行注释

C语言: <sup>②</sup> `/* ... */` → 多行注释  
通用

8.12

Date

```

if (条件成立) {
    ...
} else {
    ...
}

```

else 表示“否则的话”。

“{ }”也可以不用，但“;”不加在 if (...) 后，  
应将“分号;”加在 if (...) 整个语句末。

if - else 结构中 要么执行 if 后的程序，要么执行 else 后的程序  
两者只取其一执行。

另：else 总是与其最近的 if 配对（除了有大括号的情况）  
要始终在 if (...) 与 else 后加上“{ }”，习惯。

级联的 if - else if - else → 层层递进，更优写法

switch - case

```

switch (控制表达式) {
    case 常量:
        语句...
    case 常量:
        语句...
    :
    default:
        语句...
}

```

区别于 if 语句：

if 语句是一句句走下来直到符合 if 条件；  
switch case 语句会直接走到相应 case。  
更具效率。

必须是 int 型变量

“常量” < 数值  
数值组成的计算式。

（“case”只是“路牌”，  
不阻止程序）

若没有 break 程序会一直往下走。

（中间可用“break”终止）。（注意冒号）

8.13

Date

## 循环体 while

eg: 计算数值是几位数

```
while (x > 0) {  
    n++;  
    x = x / 10;  
}
```

注: 可以在循环体内加 `printf("x, n")`  
显示每次执行后, x 与 n 的值.

do-while: 进入循环不先做检查,  
而是先执行一次, 再开始检查是否继续执行。

```
do  
{  
    <循环体语句>  
} while (<循环体语句>);
```

生成随机数 `rand()`

将 sum 写成 `(1.0 * sum)`  $\Rightarrow$  要成浮点数计算。

输出时保留特定位小数: (double 型)

`printf("%.nlf", ...)` // 保留 n 位小数 `printf("%.nlf%%", ...)`

语言中不用 "and", 而是用 "&&" 表示 "and"

赋值时, eg: `pi = 3.1415926...`  
均有空格!!

百分数  
保留 n  
位小数

是直接接在  
result 后加了个  
%

保留小数输出  
会进行四舍五入。

8.14

△ 输出时保留特定位小数: (float 型).

printf ("%f", ...) // 保留n位.

△ 表示逻辑或者 (or): ||

8.16

整数逆序输出

```
① while (x > 0) {  
    digit = x % 10; // 取x的个位  
    ret = ret * 10 + digit; // ret为输出结果  
    x = x / 10; // 丢掉x的个位  
}
```

printf ("%d", ret);

for (初始动作; 每轮条件; 每轮动作)

△ For 循环 (一种计数循环, 初始化i, 在i达到某值及以前, 重复执行循环体, 每执行一轮, i也会变化如+1或-1)

```
for (i = 0; i <= 10; i++) {  
    语句... ;  
}
```

}

Tips for loops

{ 若有固定次数 — for 循环.  
 若必须执行一次 — do while 循环.  
 其他情况用 while 循环.

(所有for语句都能被改写为while语句)

第k个菲波那契数

8.17 (5.23课时)

eg: 判断 x 是否为素数

```

for (i = 2; i < x; i++) {
    if (x % i == 0) {
        isPrime = 0;
        break;
    }
}
if (isPrime == 1) {
    printf("是素数");
} else {
    printf("不是素数");
}

```

// isPrime 初始化为 1

break & continue

break: 跳出循环, 读接下来的程序。

continue: 跳过这一轮循环的剩余语句, 进入下一轮循环。

8.19

eg: 输入 k 表示第 k 个菲波那契数, 输出其值。

```

if (k == 1 || k == 2) {
    x = 1; // 第 1, 2 个数都为 1
}
int a = 1, b = 1;
for (int i = 3; i <= k; i++) {
    x = a + b;
    a = b;
    b = x;
}
printf("%d", x);
return 0;

```

8.21

① 循环读取 多组输入数据:

```
while (scanf("%d\n", &n) != EOF) {
```

```
    ...
    printf("%d\n", n); }
```

↓  
对应多组输出

② 8.23

传送门 goto

```
eg: if ( ) { ...
```

```
    ... goto out;
```

```
}
```

out: ← 有接从 "out" 后  
开始运行。

(一般用于跳出多重循环)

③ 计算  $1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} \cdots + \frac{1}{n}$ :

符号交错型,

```
int sign = 1;
```

```
for (int i = 1; i <= n; i++) {
```

```
    sum = sum + sign * 1.0 / i;
```

```
    sign = -sign; // 每循环
```

一次转变符号。

```
}
```

④ 整数正序输出

eg:  $x = 13425$

从最高位一位一位取d

$d = x / 10000$ ; // 取最高位

$x = x \% 10000$ ; // 去掉最高位

(以此类推循环.)

⑤ 最大公约数  
辗转相除法

若  $b = 0$ , 结束, a 即为 gcd(a, b).

否则, 令  $t = a \% b$ ;

$a = b$ ;

$b = t$ ; (while 循环)

最终  $\text{printf}("%d", a)$ ;

Date 8.23

### ①最小公倍数

$$\text{LCM}(a, b) = |a * b| / \text{GCD}(a, b) \triangleq$$

这样能避免代码超时。

8.24

### long long 型变量

输入输出用 "%lld"

防止数字过大而溢出。

$\left\{ \begin{array}{l} \%o - 8\text{进制} \\ \%x - 16\text{进制} \end{array} \right.$

8.26

### sizeof(....)

可用于输出变量类型  
的内存(字节)长度。

"( )"内不能做运算。

### 补码

(二进制)

(char类型)  
(8bits)

即二进制数的相反数

1  $\rightarrow$  0000 0001

哪个二进制数加上 00000001 会为 0?

答: 11111111



对于 +a, 其补码为 0-a  
实际是  $2^n - a$ ,  
n 为这种类型的  
位数

$\therefore 11111111 + 00000001$

$\Rightarrow 100000000$ , 而 "1" 会被省略,

故  $11111111 + 00000001 = 0$

若 00000001 为 "1", 11111111 即为 "-1"

8.26.

## 补码

对于8个字节的:

范围 0000 0000 ~ 1111 1111

其中 0000 0000 → 0

△ 1111 1111 ~ 1000 0000 → -1 ~ -128

0000 0001 ~ 0111 1111 → 1 ~ 127

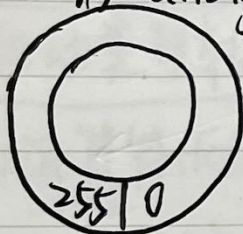
输入输出用 %u

在变量类型前 + "unsigned" 表示作 纯二进制运算

eg: unsigned char a = 255;

(0000 0000 ~ 1111 1111 → 0 ~ 255, 而非 -128 ~ 127)

有 "unsigned"



无 "unsigned"



## 整数的范围

char: 1字节, -128 ~ 127

short: 2字节, -32768 ~ 32767

int: 取决于CPU, 通常为"1字"

long: 4字节

long long: 8字节

$-2^{n-1} \sim 2^{n-1}-1$

浮点数输出若用 %e → 科学计数法

eg:  $-5.67e+03 \rightarrow -5.67 \times 10^3$ ;  $1.234e-16 \rightarrow 1.234 \times 10^{-16}$

("e" 也可用大写 "E")

无穷大, 无穷小, 或无意义的数值可用浮点数表示, 若用 int 则会报错  
(±inf) (nan)

eg:  $12.0/0.0 = \text{tinf}$ ;  $-12.0/0.0 = -\text{inf}$   
 $0.0/0.0 = \text{nan}$ .

(带小数点的数字面量均为 double.)  
在数后 + "f" 才是 float.

## 浮点数运算的精度

eg:  $1.345f + 1.123f \neq 2.468$

因此,  $f_1 = f_2$  可能失败.

应该用  $\text{fabs}(f_1 - f_2) < 1e-12$  \*  
(绝对值)

来判断  $f_1, f_2$  是否相等。

## 字符类型

8.27

char 是一种整数, 也是一种 字符类型。

printf 与 scanf 用 %c 输出输入。

用 单引号 表示字符字面量: 'a'; '1'。

(" 也是一个字符)

如何输入字符 'I' 给变量 c?

eg:  $\text{scanf}(\text{"%c"}, \&c); \rightarrow \text{I}$

$\text{scanf}(\text{"%d"}, \&i); c = i; \rightarrow 49$

explanation: 字符 "I" 的 ASCII 编码 是 49

∴ 当  $c == 49$  时, c 就是字符 'I'。

## 字符计算

No.

Date

① eg: `char c = "A";`  
`c++;`

→ `c = "B"`

② eg: `int i = "Z" - "A";`

→ `i = 25;`

(字符相减, 得到距离)

因此,  $a + 'A' - 'a'$  可将小写 `a` 变为大写 `A`。

$A + 'a' - 'A'$  可将大写 `A` 变为小写 `a`。

## 自动类型转换

当运算符两边出现不同类型时,

会自动换成较大的类型。

$\left\{ \begin{array}{l} \text{char} \rightarrow \text{short} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{long long} \\ \text{int} \rightarrow \text{float} \rightarrow \text{double} \end{array} \right.$

对于 `printf`, 小于 `int` 会被换成 `int`,  
`float` 会被换成 `double`;

`scanf` 要自己输入, `short` 用 `%hd`

## 强制类型转换: (类型)值

eg: `(int) 10.1`

不会改变原变量, 而是产生一个新的值。

eg: `int i = 32768;`

`short s = (short) i; → s = 0`

但 `i` 仍为 32768。

优先级高于四则运算。

**逗号** 的优先级最低.

一般用于 for 循环:

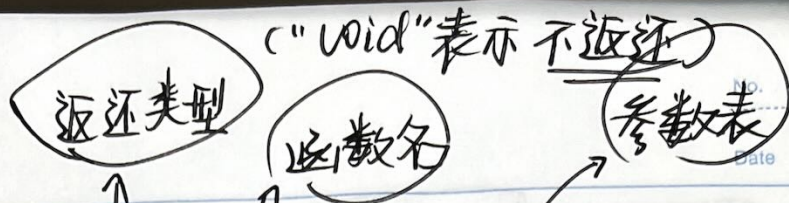
eg: `for (i=0, j=10; i<j; i++, j--)`

}

9.3

函数

eg:



void sum(int begin, int end)

函数体

{ int i; int sum = 0;

for (i = begin; i <= end; i++) {

sum += i;

} printf("%d到%d的和是%d\n", begin, end, sum);

}

调用函数: 函数名(参数值);

注: 无参数也要加 "()"。若有, 要给出正确数量和顺序的参数。(用逗号隔开)

9.5

函数是一块代码, 接收0个或多个参数, 做一件事, 并返回0个或1个值

(函数一般要先写在 int main() 的前面) 若写在后面了, 应将函数头写在前边进行声明

从函数中返回值 return 作用: 停止函数的执行, 并送回一个值

一个函数中可以使用多个 "return", 且不一定要放在最后。

传值 C语言的函数调用。

① 每个函数有自己独立的变量空间, 参数也独立于这个函数中, 与其他函数无关, 只传递数值。

② 即使变量名完全相同, 也是相互独立, 互不相同。

# 本地变量的生存期与作用域



开始?  
消亡?

什么范围  
可访问?

都是统一在“{}”内。

“{}”外定义的变量在“{}”内仍然有效。

“{}”内定义与“{}”外同名变量，则外变量

被内变量覆盖。

## 细节其他

若不写参数，void 函数名 (void) { ... }

C语言不允许嵌套函数；

int main() 也是函数；

↓  
可以不写“void”

**数组**：类型 数组名称 [数组元素个数];

eg: int sum [100];

int arr[N] 的索引是从 0 到  $N-1$ ; ☆

**初始化数组** 若 `int arr[10] = {0}`, 则剩余部分默认都为0。  
因此数组 `arr` 全部都是0。

$\Delta$  计算数组元素个数:  $\frac{\text{sizeof}(\text{数组名})}{\text{sizeof}(\text{数组名}[0])}$

↓                  ↓  
总字节长度      单元素字节长度.

9.14  
直接初始化: eg  $\begin{cases} \text{int arr}[5] = \{1, 2, 3, 4, 5\}; \\ \text{int arr}[5]; \end{cases}$

**二维数组** 存储矩阵：数据类型 数组名[行数][列数]

初始化: eg:  $\text{int arr}[2][3] = \{1, 2, 3\}, \{4, 5, 6\};$   
 (索引为第0行) (索引为第1行)

索：| 仍然都是从人日开始！

**遍历二维数组**

```
for(int i=0; i<行数; i++){
    for(int j=0; j<列数; j++){
        (scanf/printf arr[i][j])
    }
}
```

一些补充: ① `#include <math.h>` (加上头)

No.

Date

⇒ 可使用 `pow` 函数.

如 `pow(变量名, x);`

表示变量值的  $x$  次方.

② 格式化输出: `printf("%03d", ...);`

表示输出宽度为3位, 不足则在左边补0.

③ "1" 在 C 中表示 "true".

∴ `while(1) { ... }` 表示无条件一直循环,  
除非内部存在 "break"

数组冒泡排序

```
eg: for (int i = 0; i < n - 1; i++) { ①
    for (int j = 0; j < n - 1 - i; j++) { ②
        if (arr[j] < arr[j + 1]) { ③
            int temp = arr[j];
            arr[j] = arr[j + 1]; ④
            arr[j + 1] = temp;
        }
    }
}
```

①:  $n$  个数排序, 则需要  $n-1$  轮排序

②: 每一轮排序, 依次比较相邻元素  $j$  与  $j+1$

" $n-1-j$ " 是因为每排序一轮, 能排好最后一个元素, 不用再排了.

③.④ 即为按要求进行相应比较

与交换, 完成排序.

9.18

Date

① 一个字节有8比特。"int"为4字节；"double"为8字节。

② 之前 scanf 中用到的 "&" 是用来获得变量的地址，其操作数必须为 变量。

③ 获取地址用 %p  $\Rightarrow$  `printf("%p", &变量)`  
输出格式为 十六进制 (%x)

④ 若先定义一个 "i"，再定义一个 "p"。

i, p 是 相邻 存放在内存中的，

(堆栈)

如 i 的地址为 ... 6c；p 的地址则为 ... 68

i: ... 6c  
↓  
p: ... 68

先定义的，地址位置 更高，由上到下

⑤ 对于 数组，&a 与 a 与 a[0] 的地址都是相同的  
(int a[10])  
之后相邻索引地址都相差 4。

⑥ 数组作为函数参数

int f(int a[], int len) ... { ... }

需输入  
一个 数组

需输入  
数组元素个数

9.18

别的变量储存具体的值  
 指针变量储存别的变量的地址。

指针 (一种保存地址的变量)

eg: `int i;`

`int* p = &i;`

"\*"表示"`p`"为一个指针;  
 指针`p`获得`i`的地址的值,  
 即"`p`指向了`i`"。

\*注意: eg: `int* p, q;` 中:

`p`是指针, 但`q`并非指针

(\*是用于变量名上, 而非`int`上)

指针作为参数

eg: `void f(int* p);`

则若有 `int i;`

就这样调用函数:

`f(&i);`

从此, 函数内可以访问外部变量`i`的地址了

以前只能传输 `i` 的具体值

(指针) 像虫洞, 连接不同“块”内的变量。

Date

9.21

访问地址上的对应变量

使用“\*” (单目运算符)。

eg:  $\text{int} * p = \&i;$  ( $p$  储存  $i$  的地址)

则  $*p \Rightarrow i$ ;  $*p$  的值即为  $i$  的值

若  $\text{int } i = 0;$

$*p = 1;$

$\text{printf}("%d", i);$

$\Rightarrow$  得到 1 而非 0

( $p$  原本为变量的地址,  
但“ $*p$ ”就约等于变量本身)  
 $\Delta$

9.22

(注意: 定义了指针, 例如  $\text{int} * p;$  若  $p$  还未指向任何变量,  
则不能使用“ $*p$ ”的赋值等操作。)

9.23

数组与指针 eg:  $\text{int } f(\text{int } a[], \text{int } len)$

① 函数参数表中的“ $a[]$ ”数组, 实际上是指针  $\Delta$

② 数组变量是特殊的指针。

(本身表达地址)

③ 数组变量整体:  $a[n] \Rightarrow \text{int} * p; p = a$  // 需用“&”  
④ 数组单个索引:  $a[0] \Rightarrow \text{int} * p; p = \&a[0]$  // 需用“&”

④ 数组变量是 const 的指针, 因此不能被赋值  $\Delta$

(常量)

# 指针与 const

① 指针是 const:

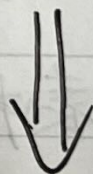
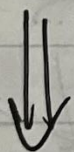
int \* const p; // p 是 const  
= &i

⇒ { \*p = 26 // ok  
    p++ // error

② 所指是 const:

const int \*p = &i; / int const \*p = &i;

⇒ { i = 26 // ok  
    p = &j // ok  
    \*p = 26 // error.  
            (\*p 是 const)



然后, 因为将数组作为参数传入函数时,  
传入的是指针(地址).

因此函数内部可能修改数组的值

若要保护外部数组不被破坏,

可以 int f(const int a[], int len)

9.25

指针运算

1. "p++" (p 为一个指针):

$$\begin{cases} \text{int}^* p = a[n]; \text{"p++"} \text{ 实际上地址移动了 } \text{sizeof}(\text{int}). \\ \text{char}^* p = a[n]; \text{"p++"} \text{ 地址移动了 } \text{sizeof}(\text{char}). \end{cases}$$

↓

↓

↓

eg:  $\text{int}^* p = a[10];$

则  $*(p+1) == a[11];$

$*(p+n) == a[n];$

2. 两个指针相减 9.26

eg:  $\text{int}^* p = a[1]; \text{int}^* q = a[5];$

$q - p = 4$  (指 q 与 p 之间还有 4 个 int 型的值)

3.  $*p++$ : 指, 先取出 p 所指向的值, 再将 p 移到下一个单位

4. 指针比较大小 eg:  $\text{int}^* p = a[0]; \text{int}^* q = a[1];$  则  $q > p$ .

5. 0 地址: 用 "NULL" 表示。

6. 指针的类型: 无论 p 指向的是什么类型的变量,  
指针大小都一样, 因为都是地址。

但是,

指向不同类型的指针不能互相赋值。

7.  $\text{void}^* p$  表示不知道指向什么的指针。

# 动态内存分配

## 1. malloc函数:

需头文件 `#include <stdlib.h>`

eg: `int *a;`

`a = (int*) malloc(n * sizeof(int))`

① 向 malloc 申请空间的大小是以字节为单位的。

② 返回结果是 void\*，需自己转换类型。

之后，指针a完全可以当作数组使用

借完空间后要归还: `free(a)`

只能归还申请空间的首地址

(eg: 若 `a++` 了，不能再归还 `free(a)`，应 `free(a-1)`)

0 或 '\0' (但 '0' 不行)

No.

Date

10.1

字符串 = 以 整数0 结尾的一串字符

注: ① 0 标志字符串的结束, 但 0 不是字符串的一部分.  
计算字符串长度不包含 0

② 字符串以 数组 或 指针 形式存在.

可用 数组, 指针 的形式访问

③ <string.h> 有很多处理字符串的函数

2. 字符串变量 (字符串是数组)

eg:  $\begin{cases} \text{char}^* \text{str} = \text{"Hello"}; \\ \text{char word}[ ] = \text{"Hello"}; \\ \text{char line}[10] = \text{"Hello"}; \end{cases}$

用 双引号 表示  
字符串变量

(在 "line" 内占用 5+1 → 6 个空间.  
因为末尾还有一个 0)

3. 字符串常量

eg:  $\text{char}^* \text{s} = \text{"Hello, world"};$

指针 s 指向的字符串常量, 其所在的地方是 "代码段",  
只读不写, 即, s 其实是 const char\* s

(即不能用  $\text{SEI}[ ] = \dots, \text{SEI}[ ] = \dots,$   
来改变它的值)

使用字符串时: 用指针 or 数组定义?

$\left\{ \begin{array}{l} \text{char} * s = \text{"Hello"}; \text{ 作参数, 动态分配空间.} \\ \text{char word}[1] = \text{"Hello"}; \text{ 能用索引修改.} \end{array} \right.$

10.2

字符串输入输出 (%s)

$\left\{ \begin{array}{l} \text{scanf}(\text{"\%s"}, \text{string}); \text{ (不用 \&, 字符串本身是数组)} \\ \text{printf}(\text{"\%s"}, \text{string}); \end{array} \right.$

scanf 读入一个单词, 到空格, tab 或回车为止。

scanf 时, 用 "%ns" 表示最多读入 n 个字符

若有多余的字符没有读入, 剩余的会交给下一个 scanf

空字符串

$\text{char buffer}[100] = \text{""};$

$\left\{ \begin{array}{l} \text{空字符串, } \text{buffer}[0] = \text{"\0"}; \text{ 或 } \text{"\0"}; \end{array} \right.$

$\text{char buffer}[1] = \text{""};$

这个数组长度则只有 1, 即  $\text{buffer}[0]$ .

10.2

字符串数组①  $\Delta$   $\boxed{\text{char a[I][n]} = \{ "...", "...", \dots \}};$ 即  $a[0]$  是一个  $\text{char}[n]$ ,  $a[1]$  又是另一个  $\text{char}[n]$ ...  
(字符串) 另②  $\Delta$   $\boxed{\text{char *a[I]} = \{ "...", "...", \dots \}};$ 即指针  $a[0]$  指向一个字符串,  $a[1]$  又指向另一个字符串...

10.3

字符串函数 (需 + 头文件  $\langle \text{string.h} \rangle$ )① strlen → 返回字符串长度, 不包含末尾 '\0'.eg:  $\text{char word[I]} = \text{"Hello"};$   
 $\text{strlen(word)} = 5.$ ② strcmp → 比较两个字符串的大小

负数

0 → 两个字符串相同.

正数

eg:  $\begin{cases} \text{char s}_1[\text{I}] = \text{"abc"}; \\ \text{char s}_2[\text{I}] = \text{"bbc"}; \end{cases}$   
 $\text{strcmp(s}_1, \text{s}_2) = -1; (\because 'a' - 'b' = -1)$ ③ strcpy 将一个字符串复制给另一个字符串 (空间) $\text{strcpy(dst, str)}$  (但两个字符串地址不能有重叠)(会返回一个  $\text{dst}$ )

eg: `char* dst = new (char*) malloc (strlen(src)+1);`

`strcpy(dst, src);`

先申请空间(记得+1), 再做复制

④ strchr 字符串中找字符(第一次出现)(返回位置)

{ 从左往右找: `char* strchr(const char* s, int c);`  
 从右往左找: `char* strrchr(const char* s, int c);`

该函数返回的是一个指针△

eg: `char s[] = "hello";`  
`char* p = strchr(s, 'l');`  
`printf("%s", p);`

⇒ `llo`  
 (finish in...)

此时 p 是 "llo"  
 而 \*p 就为 'l'  
 ☆

若想得到 "hello" 中的 "he",

(可将 \*p = '\0'; 这样 

|   |   |    |   |   |   |
|---|---|----|---|---|---|
| h | e | \0 | l | l | o |
|---|---|----|---|---|---|

,

S 就变成 "he" 了, 再使用 strcpy 获得 "he"

⑤ strstr 在字符串中找字符串

strcasestr 忽略大小写  
 在字符串中找字符串

不好用哈！

No.

Date

**枚举** 一种用户定义的数据类型

① enum 枚举类型名称 {名字0, 名字1, ...};

都是一些常量符号, 值依次从0到n.  
用来 replace,  
const int red = 1; 这种.

作为参数时, 要在参数表内写(enum 类型)(常量)  
但实际上都可以是int型.

② 自动计数的枚举

enum color {red, yellow, green, numcolors};

↓

char \* colornames [numcolors] = {"red", "yellow",  
"green"};

↓  
用来初始化字符串数组

10.6

① 实现数据(浮点数)的四舍五入 =  $(int)(x+0.5)$

一些函数

② #include <ctype.h>  
c头文件

isalnum() 判断字符是否是数字或字母  
是 → 返回非零值  
否 → 返回0

isalpha() 判断是否是字母

isdigit() 判断是否是数字

10.9

① 万能头文件: #include <bits/stdc++.h> ★  
using namespace std;  
(用于C++, 一般不在C里写)

② getchar & putchar 函数

输入 : 输出.

int getchar(void)

返回读取的字符(int型)

失败或达到文件尾会返回EOF  
会等待用户输入直到按回车.

int putchar(int char)

每次输出一字符

失败会返回EOF

读取多字符: while((c=getchar())!='\n'){...}

总结, eg.:

char a=getchar();

putchar(a);

⇒